Classifying Unidentified Payments
AI Experiment

An AI-Powered Payment Classification Pipeline

CLIENT

Department of Cornell University Treasury

TEAM

Sanjeev Ragunathan(sr2432) · Sruti Sri-Sai Gudapati(ssg98) · Yaqi Liu(yl3879)

Project Advisor: Ayham Boucher

REPOSITORY

https://github.com/sanjeev-ragunathan/cornell-treasury-automation

Cornell University · AI Innovation Lab · Spring 2026

Abstract

Cornell University Treasury receives hundreds of incoming electronic payments daily, many of which arrive without clear indication of which Cornell department should receive the funds. Identifying the correct department currently requires manual investigation that can take 30 to 40 minutes per payment. We designed and built an automated classification pipeline that ingests EDI 820 Excel reports exported from Kyriba and outputs a reviewable Google Sheet of department classifications in seconds rather than hours. The system combines a deterministic ranking algorithm using historical General Ledger (GL) transaction data with an LLM-based classifier (GPT-4o). A reconciliation layer merges both signals, applies fallback rules, and flags uncertain classifications for human review. This report describes the problem context, technical approach, evaluation results, limitations, and the migration path to a Claude Code Skill running within Cornell's secure AI infrastructure.

1. Problem and Stakeholder Context

Cornell University Treasury handles incoming electronic funds including ACH transfers, wire payments, and other deposits. These payments arrive as EDI 820 (Payment Order / Remittance Advice) messages, which Treasury exports from Kyriba as Excel reports.

A significant portion of these payments arrive with vague, truncated, or missing information. Treasury staff must manually determine which department should receive the funds by searching vendor names, reviewing historical records, and contacting departments across Cornell. A single payment can take 30 to 40 minutes to resolve, translating to roughly 150 hours of work for a batch of 282 unidentified payments.

Our stakeholder for this project was the Department of Cornell University Treasury, specifically the operations team responsible for processing incoming payments. Our project advisor was Ayham Boucher.

2. Approach

We framed the problem as a classification task: given an incoming payment record, predict which Cornell department should receive the funds while also estimating confidence and whether the result requires human review.

Rather than relying entirely on an LLM, we used Cornell Treasury's historical General Ledger transaction data, over 101,000 records, as the primary signal. Our solution combines two complementary approaches:

- A deterministic ranking engine that compares incoming payments against cleaned historical records using payer similarity, amount matching, reference IDs, keyword overlap, and generic-pattern penalties.
- An LLM (GPT-4o) that receives the payment text along with the top-ranked historical matches as structured context and produces a final classification.

A reconciliation layer merges both outputs, applies fallback rules, computes evidence strength, and flags uncertain records for manual review. Final classifications are written to a Google Sheet that Treasury staff can filter and review efficiently.

We implemented the pipeline in n8n using Python code nodes for scoring, prompt formatting, and reconciliation logic. Google Sheets served as both the source of historical data and the destination for classified output.

3. System Architecture

The pipeline consists of two workflows:

Daily History Cache Refresh

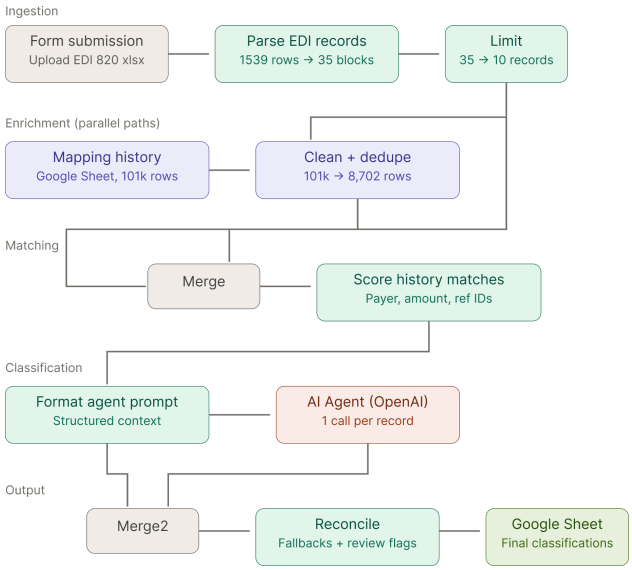
A scheduled workflow runs once per day and processes 101,000+ historical GL transaction records from Google Sheets. The workflow cleans and deduplicates the data, normalizes fields, removes overly generic patterns, and creates a smaller cache of approximately 8,700 high-quality historical rows. Caching significantly reduces classification runtime.

Payment Classification

Treasury staff upload an EDI 820 Excel file through a form interface. The system parses the file into payment records, loads the cleaned history cache, and scores each payment against historical data. The top matches are passed into GPT-4o as structured context, and the resulting classifications are reconciled into a final output sheet containing:

- predicted department,
- confidence,
- evidence strength,
- and a needs_review flag.

The workflow architecture allowed rapid iteration while integrating Google Sheets, OpenAI, and custom Python logic without additional infrastructure overhead.



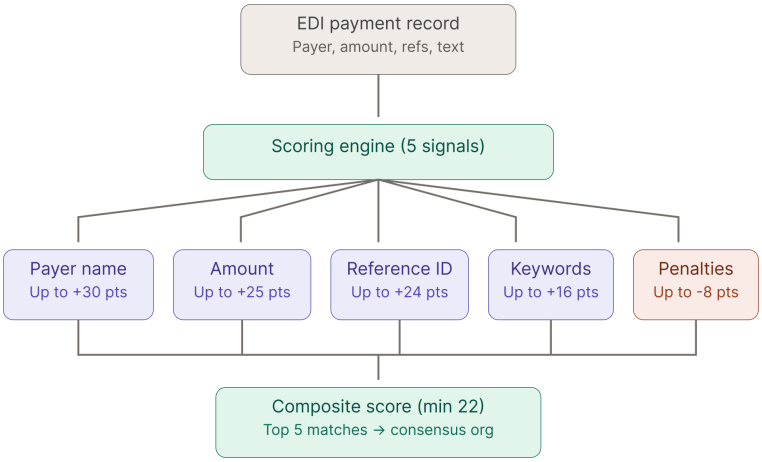
4. The Ranking Algorithm

The scoring engine is the core of the system. For each incoming payment, it compares the payment against historical transaction records using a two-pass filtering process.

The algorithm combines five primary signals:

- payer name similarity,
- amount matching,
- reference ID matching,
- keyword overlap,
- and penalties for overly generic historical matches.

Several safeguards were added to reduce false positives. Amount-only matches are rejected and generic historical rows are penalized. The top matches are then combined into a weighted consensus department prediction, which becomes the strongest historical signal passed into the LLM.



5. Reconciliation and Output

The reconciliation layer is the final processing step before results are written to the output sheet. It parses the AI output, sanitizes department names, applies fallback logic when necessary, and combines AI predictions with historical consensus signals.

Each payment receives:

- a predicted department,
- a confidence score,
- an evidence strength label,
- and a needs_review flag.

Records with weak evidence, low confidence, or unclear classifications are flagged for manual review. This allows Treasury staff to focus attention only on uncertain cases while trusting the system on straightforward matches.

6. Evaluation

We evaluated the system using real EDI 820 files provided by Cornell Treasury. The evaluation focused on payer identification accuracy, department classification accuracy, and the reliability of the needs_review flag.

The system performed well when strong historical matches existed, with the ranking engine and LLM reinforcing each other effectively. More difficult cases included unknown vendors and vendors associated with multiple Cornell departments, where exact-match accuracy was lower.

Importantly, the system reliably flagged uncertain classifications for review rather than confidently returning incorrect answers. The review flag proved highly valuable because it allowed the system to fail safely while still significantly reducing manual workload.

The scoring engine also improved through iterative refinement as we identified

failure modes and adjusted thresholds, penalties, and filtering logic.

7. Limitations

Several limitations remain in the current prototype. Treasury staff must still manually export EDI files from Kyriba and upload them into the system, since there is no direct integration with Cornell financial systems. The system also performs less accurately on unknown vendors and vendors associated with multiple Cornell departments because historical transaction signals are weaker or ambiguous in those cases. In addition, the current n8n workflow processes AI calls sequentially, which increases runtime for larger payment batches. Confidence scores produced by the LLM are also not formally calibrated against a labeled evaluation dataset. Finally, the current implementation relies on external services such as n8n cloud and Google Sheets, making the project a proof of concept rather than a production-ready deployment within Cornell infrastructure.

8. Lessons Learned

One of the most important lessons from this project was that deterministic matching should happen before involving the LLM. Historical transaction data provided a much stronger signal than raw payment text alone and greatly reduced hallucination risk.

We also learned that prompt formatting matters significantly. Converting historical matches into clean, structured text improved the LLM's reasoning quality and consistency.

Another major takeaway was the importance of designing for human reviewers rather than full automation. The needs_review flag and evidence strength tiers became some of the most valuable features because they made the system transparent about uncertainty.

9. Next Steps and Migration to Claude Code Skills

The most important next step for this project is the migration from n8n to a Claude Code Skill running on Cornell's AI Gateway. This transition addresses several limitations simultaneously by moving the system onto Cornell-approved infrastructure, reducing reliance on external services, and significantly lowering operational cost. The Claude Skill preserves the same core algorithm developed in the n8n workflow, including the ranking engine, prompt structure, reconciliation

logic, and review flagging system.

One major improvement is creating a feedback loop in which Treasury staff corrections are automatically added back into the historical mapping dataset, allowing the system to improve over time. Future work also includes calibrating confidence scores using a labeled evaluation dataset, integrating Outlook email search to provide additional context for unknown vendors, and replacing manual EDI uploads with direct Kyriba integration for near real-time processing. Finally, the system could be improved to better handle vendors associated with multiple Cornell departments by presenting reviewers with ranked department options rather than a single predicted classification.

10. Conclusion

This project demonstrates that a hybrid approach combining deterministic matching with LLM classification can meaningfully accelerate a real, costly, manual workflow at Cornell Treasury. The system reduces per-payment classification time from 30 to 40 minutes to seconds for the cases it handles confidently, while honestly flagging the cases that still require human judgment. The five-signal scoring engine, structured prompt formatting, and reconciliation logic developed in n8n form the algorithmic foundation for the production Claude Code Skill that is now being deployed within Cornell's secure AI infrastructure. The pattern of deterministic matching first, LLM as second opinion, conservative review flagging generalizes well beyond this specific problem and could be applied to other classification tasks where institutional historical data exists and accuracy matters more than autonomy.